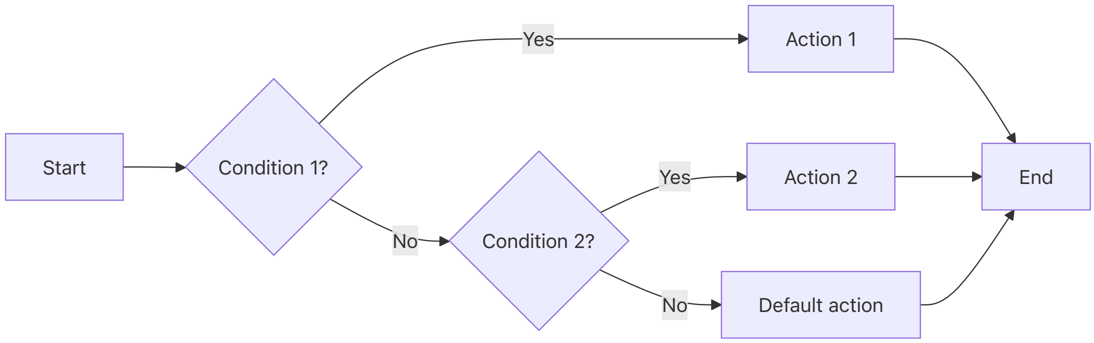
Conditional Statements (`if` , `elif` , `else`)

Conditionals allow to choose between options or take different paths.

Conditional Logic (if / else)

▶

Multiple Conditions (if / else if / else)



In [...

```python
temperature = 25

if temperature > 30:
    print("It's a hot day!")
else:
    print("It's a bit chilly.")
```

```
It's a bit chilly.
```

In [...

```python
temperature = 15

if temperature > 30:
    print("It's a hot day!")
elif temperature > 20:
    print("It's a nice day.")
else:
    print("It's a bit chilly.")
```

```
It's a bit chilly.
```

In [...

```python
x = 25
```

```python
if x >= 50:
    print("A")
elif x >= 25:
    print("B")
else:
    print("C")
```

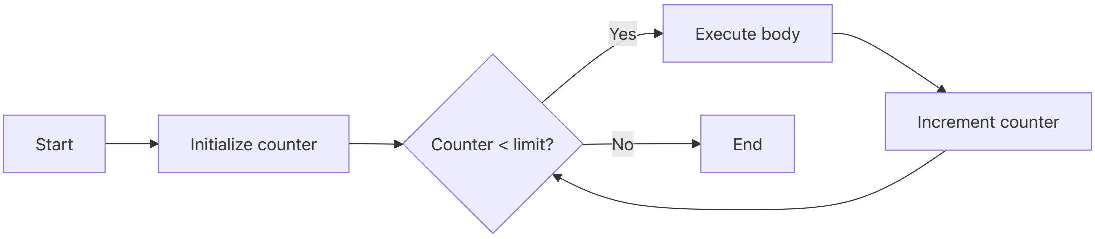
B

For Loops

Use a `for` loop when you know how many times you want to run a block of code.

Note that while the underlying model is the same, Python `for` loops works a bit differently: they are controlled by iterators.

```python
for x in iterable:
    body
```



In [...

```python
t1 = (1,2,3)
# a = t1[0]
# b = t1[1]
a, b, c = t1
print(a)
print(b)
```

1
2

In [...

```python
lst = ['a','b','c']
another_lst = ['e','f','g']

for i in lst:
    print(i)

# # i = 0
# # for elem in lst:
# #     print(elem+another_lst[i])
```

```
# #     i = i+1

# for i, elem in enumerate(lst):
#     print(elem+another_lst[i])
```

```
a
b
c
```

In [...

```
print(list(range(5)))
```

```
[0, 1, 2, 3, 4]
```

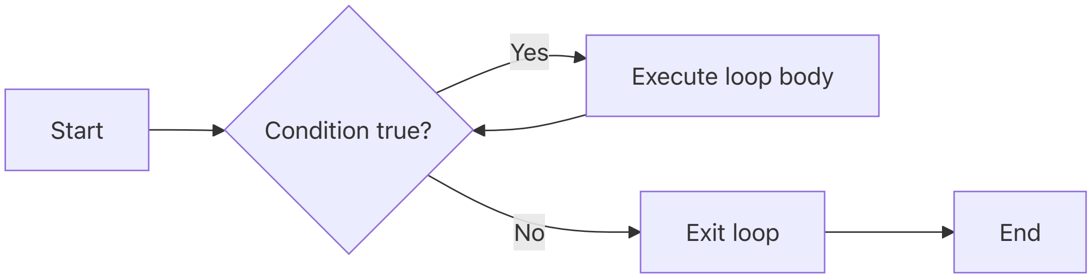
In [...

```
print("Counting to 5:")
for i in range(1, 6):
    print(i)

print("\nIterating through a list:")
colors = ["Red", "Green", "Blue"]
for color in colors:
    print(f"Color: {color}")
```

```
Counting to 5:
1
2
3
4
5

Iterating through a list:
Color: Red
Color: Green
Color: Blue
```

While Loops

Use a `while` loop when you want to repeat code as long as a condition is **True**.



In [...

```
x = 3

while x > 1:
    print('wow')
    print(x)
    x = x -1
```

```
wow
3
wow
2
```

In [...

```
energy = 3
while energy > 0:
    print(f"Running... Energy left: {energy}")
    energy -= 1
print("Stopped. Need to rest!")
```

```
Running... Energy left: 3
Running... Energy left: 2
Running... Energy left: 1
Stopped. Need to rest!
```

In [...

```
# while True:
#     print('wow it never stops')

# while False:
#     print('such sadness')
```

for and while usecases:

- `for` : Use when you want to repeat some action for a known number of times, while iterating over an iterator. Example: processing a **known** number of files in a folder.
- `while` : repeat some action for an unknown number of times, based on some condition.

More: Check this page from Real Python

Break and Continue

- `break` exits the loop early.
- `continue` skips the current iteration and moves to the next.

In [...

```
print("Testing Break (stops at 3):")
for n in range(1, 6):
    if n == 3:
        break
    print(n)

print("\nTesting Continue (skips 3):")
for n in range(1, 6):
    if n == 3:
        continue
    print(n)
```

```
Testing Break (stops at 3):
1
2

Testing Continue (skips 3):
1
2
4
5
```

In [...

```
for n in range(6):
    # break
    print(n)
    if n == 3:
        break
        # continue
```

```
0
1
2
3
```

In [...

```python
x = 'srm university-ap'
vowels ='aeiou'
# vowels = ['a', 'e', 'i', 'o', 'u']
for i in x:
    if i in vowels:
        print(i)
    continue
```

```
u
i
e
i
a
```

note: pass and continue are not same in python! more here: https://stackoverflow.com/questions/9483979/is-there-a-difference-between-pass-and-continue-in-a-for-loop-in-python

In [...

```python
# one example why all these is going to be useful
filenames = []
for i in range(1,3):
    filenames.append(f'excel{i}')

import pandas as pd
for filename in filenames:
    foo = pd.read_excel(f'{filename}.xlsx')
    foo['product'] = foo['A']*foo['B']
    foo.to_excel(f'updated_{filename}.xlsx',index=Non
```

In [...

```python
filenames = []
for temp in range(1,3):
    if (temp>1):
        filenames.append(f'excel{temp}.xlsx')
print(filenames)
for filename in filenames:
    _ = pd.read_excel(filename)
    print(_)
# print(filenames)
[f'excel{i}.xlsx' for i in range(1,20,2) if i%2==1]
```

```
['excel2.xlsx']
   A   B
0  4  11
1  5   2
2  6   3
```

Out[…

```
['excel1.xlsx',
 'excel3.xlsx',
 'excel5.xlsx',
 'excel7.xlsx',
 'excel9.xlsx',
 'excel11.xlsx',
 'excel13.xlsx',
 'excel15.xlsx',
 'excel17.xlsx',
 'excel19.xlsx']
```

In […